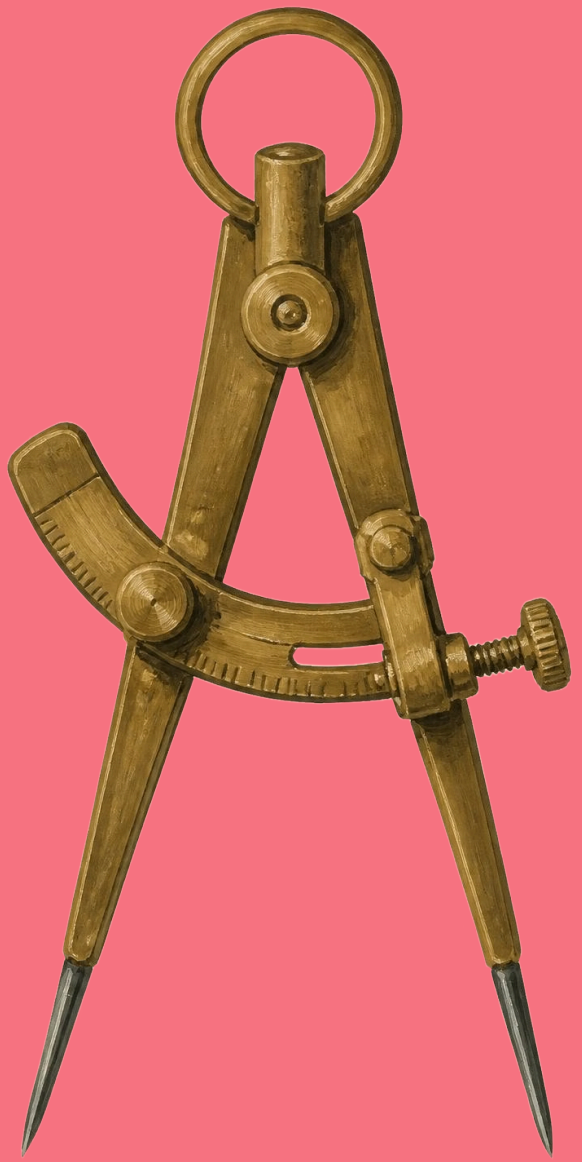
DESIGN AND ANALYSIS OF ALGORITHMS · DAA

Master Theorem in Algorithm Analysis



PRESENTED BY

Nirmayee S. Kadam

PRN: 25070521134

ACADEMIC DETAILS

Sem-III (C) · CSE

SIT, Nagpur

Made with GAMMA

CORE PURPOSE

Solving divide-and-conquer recurrences

01 Solves recurrence relations directly

Avoids unrolling long algebraic expansions or drawing full recursion trees manually.

02 Determines asymptotic time complexity

Yields tight asymptotic bounds Θ for standard divide-and-conquer algorithms.

03 Simplifies recursive algorithm analysis

Provides an instant cookbook method for standard benchmarks like Merge Sort and Binary Search.



Made with GAMMA

STANDARD FORMULATION

General recurrence formula

RECURRENCE RELATION

$$T(n) = aT(n/b) + f(n)$$

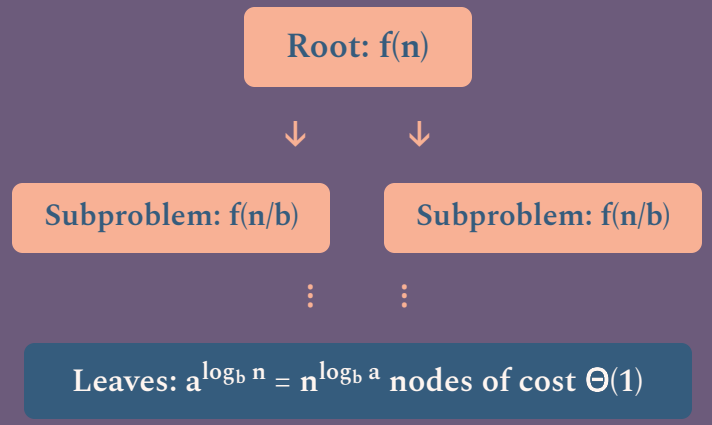
$a \geq 1$: number of recursive subproblems

n/b $(b > 1)$: size of each subproblem

$f(n)$: cost of dividing and combining solutions

$T(n)$: total time complexity on input size n

RECURSION TREE DECOMPOSITION



Total depth = $\log_b n$ levels

Made with GAMMA

CRITICAL COMPARISON

Three cases of the Master Theorem

Let p = log_b a. Here ε > 0 and k ≥ 0.

CASE 1

The Leaf Work Dominates

Condition: $f(n) = O(n^{\log_b a - \epsilon})$ for some constant ε > 0; non-recursive work f(n) is polynomially smaller than n^{log_b a.

Result: $T(n) = \Theta(n^{\log_b a})$

Most of the algorithm's time is spent at the bottom leaf level of the recursion tree. [1, 2, 3, 4]

CASE 2

The Work is Balanced Across Levels

Condition: $f(n) = \Theta(n^{\log_b a} \log^k n)$ for some constant k ≥ 0.

Result: $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$

Work is distributed evenly across recursion-tree levels; multiply the work at a single level by tree height (≈ log n). [1, 2, 3]

CASE 3

The Root / Non-Recursive Work Dominates

Condition: $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant ε > 0, and the regularity condition: $af(n/b) \leq cf(n)$ for some constant c < 1, for all sufficiently large n. Master Theorem: Practice Problems and Solutions.

Result: $T(n) = \Theta(f(n))$

Most algorithm time is spent at the root level doing f(n) outside the recursive calls. [1, 2, 3]

Made with GAMMA

STEP-BY-STEP DERIVATION

Worked example: Merge Sort

$$T(n) = 2T(n/2) + n$$

1. Extract parameters: $a = 2,\ b = 2,\ f(n) = n$

2. Compute critical exponent: $\log_b a = \log_2 2 = 1$

3. Compare terms: $n^{\log_b a} = n^1 = n$

4. Match case: $f(n) = \Theta(n^1 \log^0 n)$ (Case 2 with $k = 0$)

Final Complexity: $T(n) = \Theta(n \log n)$

MERGE SORT RECURSION TREE

Level	Work
Level 0	Root size n; cost n
Level 1	2 subproblems of size $n/2$; $2 \times (n/2) = n$
Level 2	4 subproblems of size $n/4$; $4 \times (n/4) = n$
$\log_2 n$	n leaves of cost $\Theta(1)$; $n \times \Theta(1) = n$

Total Work = (Cost per Level: n) × (Tree Height: $\log_2 n$) = $\Theta(n \log n)$

Made with GAMMA

PRACTICAL SCOPE

Applications and limitations

PRIMARY APPLICATIONS

Merge Sort

$T(n) = 2T(n/2) + \Theta(n) \implies \Theta(n \log n)$

Binary Search

$T(n) = T(n/2) + \Theta(1) \implies \Theta(\log n)$

Strassen's Matrix Multiplication

$T(n) = 7T(n/2) + \Theta(n^2) \implies \Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$

KNOWN LIMITATIONS

Non-constant partitioning

Does not apply when subproblem sizes are unequal (e.g., $T(n) = T(n/3) + T(2n/3) + n$ in Quick Sort).

Polynomial gaps fail

Gaps must be strictly polynomial: cannot solve $T(n) = 2T(n/2) + n/\log n$.

Alternative techniques required

Must fall back to substitution methods, recursion trees, or the Akra-Bazzi method.

Made with GAMMA

TAKEAWAYS & WRAP-UP

Conclusion

Standardized recurrence analysis

Replaces tedious inductive algebraic work with a straightforward parameter comparison.

Categorized problem taxonomy

Classifies recurrence behavior into leaf-dominated, balanced, or root-dominated cases.

Essential engineering tool

Enables rapid estimation of asymptotic efficiency and architectural scalability.

Thank you!



Made with GAMMA